# SkyCast

Application de gestion météo – Architecture microservices Spring Boot

| Information | Détail |
|---|---|
| Classe | *4DAS_I17* |
| Membres de l'équipe | Maissa Messoussi, Alaa Ben Moussa, Mariem Hamdi, Maryem Zayani, Aziz Tombari |
| Enseignant | Alaa RAMI |
| Dépôt Git | *[lien ]* |
| Année universitaire | *2026 / 2027* |

# Sommaire :

**1. Introduction et description du sujet....4**
1.1 Contexte.... 4
1.2 Problématique....4
1.3 Objectifs et périmètre.... 4
1.4 Équipe....4
**2. Acteurs et fonctionnalités....4**
2.1 Acteurs....4
2.2 Diagramme de cas d'utilisation.... 5
2.3 Fonctionnalités par acteur et service responsable....6
**3. Architecture globale.... 7**
3.1 Diagramme d'architecture globale.... 7
3.2 Composants....7
3.3 Principes architecturaux....8
**4. Découpage en microservices.... 8**
4.1 Justification des choix de bases de données.... 8
4.2 Répartition des services.... 8
**5. Conception détaillée des microservices.... 9**
5.1 Ville Service.... 9
Données gérées et base de données.... 9
Contrat d'API.... 9
Événements.... 9
Interactions avec les autres services....9
5.2 Station Service....10
Données gérées et base de données.... 10
Contrat d'API.... 10
Événements.... 10
Interactions avec les autres services....10
5.3 Relevé Service....10
Données gérées et base de données.... 10
Diagramme de classes....11
Contrat d'API.... 11
Événements.... 11
Interactions avec les autres services....11
5.4 Prévision Service....11
Données gérées et base de données.... 11
Diagramme de classes....11
Contrat d'API.... 11
Événements.... 12
Interactions avec les autres services....12
5.5 Alerte Service.... 12

Données gérées et base de données..... 12
Diagramme de classes.....12
Contrat d'API..... 12
Événements..... 12
Interactions avec les autres services.....12
5.6 Service commun (Auth).....13
Données gérées et base de données..... 13
Diagramme de classes.....13
Contrat d'API..... 13
Événements..... 13
Interactions avec les autres services.....13
**6. Interactions entre services.....13**
6.1 Communications synchrones.....13
6.2 Communications asynchrones.....14
**7. Sécurité..... 14**
**8. Technologies choisies.....14**
**9. Organisation du dépôt Git et gestion de projet..... 15**
9.1 Structure du dépôt..... 15
**10. Conclusion et perspectives.....15**
**Annexe A – Diagramme de classes d'analyse global..... 16**

# 1. Introduction et description du sujet

## 1.1 Contexte

Les données météorologiques (températures, humidité, vent, pression) sont collectées par des stations équipées de capteurs, puis exploitées pour produire des historiques, des statistiques, des prévisions et des alertes. L'application de gestion météo centralise ces informations par ville et les met à disposition de trois catégories d'utilisateurs : les visiteurs, les utilisateurs inscrits et les administrateurs.

## 1.2 Problématique

Comment centraliser et exposer des données météo fiables et à jour, tout en prévenant rapidement les utilisateurs abonnés en cas de phénomène dangereux (canicule, froid, tempête, fortes pluies, inondation), au moyen d'une architecture évolutive et faiblement couplée ?

## 1.3 Objectifs et périmètre

- Proposer une consultation publique des villes, prévisions, dernières mesures et alertes actives.
- Permettre aux utilisateurs authentifiés de s'abonner aux villes, de choisir leurs alertes et de consulter historiques et statistiques.
- Offrir aux administrateurs la gestion complète des données (villes, stations, capteurs, mesures, prévisions, règles, alertes, comptes).
- Mettre en œuvre une architecture microservices avec API Gateway, découverte de services, sécurité JWT et communication synchrone et asynchrone.

Ce document correspond à la validation intermédiaire : il présente la conception ; l'implémentation n'est pas encore finalisée.

## 1.4 Équipe

| Membre | Microservice responsable |
|---|---|
| Maissa Messoussi | Alerte |
| Alaa Ben Moussa | Ville |
| Mariem Hamdi | Station |
| Maryem Zayani | Prévision |
| Aziz Tombari | Relevé |

Le service commun (User) est développé par toute l'équipe.

# 2. Acteurs et fonctionnalités

## 2.1 Acteurs

| Acteur | Authentification | Description |
|---|---|---|
| Visiteur | Non | Consulte les informations publiques sans compte. |

| Acteur | Authentification | Description |
|---|---|---|
| Utilisateur (USER) | Oui | Hérite des droits du visiteur et accède aux fonctions personnalisées. |
| Administrateur (ADMIN) | Oui | Gère l'ensemble des données et des comptes de la plateforme. |

## 2.2 Diagramme de cas d'utilisation

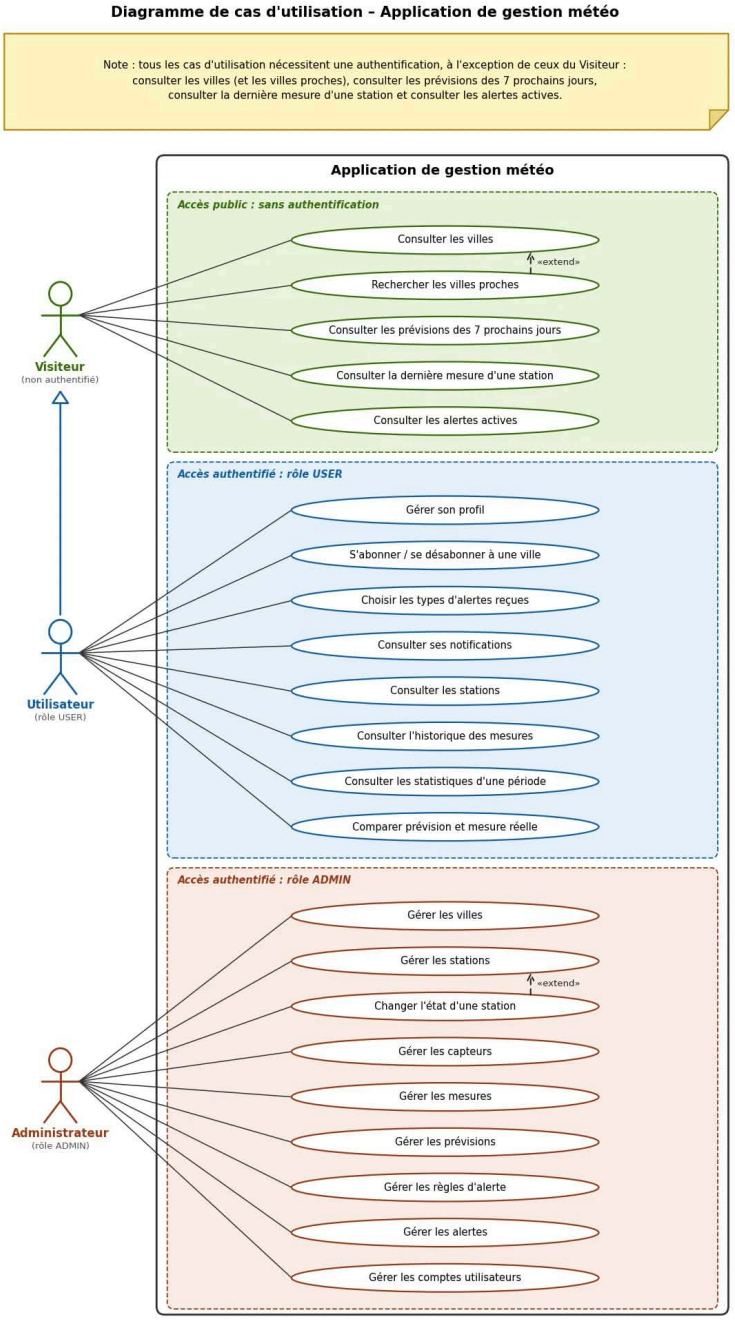



*Figure 1 – Diagramme de cas d'utilisation*

## 2.3 Fonctionnalités par acteur et service responsable

| Acteur | Fonctionnalité | Microservice |
|---|---|---|
| Visiteur | Consulter les villes et rechercher les villes proches | Ville |
| Visiteur | Consulter les prévisions des 7 prochains jours | Prévision |
| Visiteur | Consulter la dernière mesure d'une station | Relevé |
| Visiteur | Consulter les alertes actives | Alerte |
| USER | Gérer son profil | Commun (Auth) |
| USER | S'abonner / se désabonner à une ville ; choisir les types d'alertes | Alerte |
| USER | Consulter ses notifications | Alerte |
| USER | Consulter les stations | Station |
| USER | Consulter l'historique et les statistiques des mesures | Relevé |
| USER | Comparer prévision et mesure réelle | Prévision (+ Relevé) |
| ADMIN | Gérer les villes | Ville |
| ADMIN | Gérer les stations, capteurs et changer l'état d'une station | Station |
| ADMIN | Gérer les mesures | Relevé |
| ADMIN | Gérer les prévisions | Prévision |
| ADMIN | Gérer les règles d'alerte et les alertes | Alerte |
| ADMIN | Gérer les comptes utilisateurs | Commun (Auth) |

# 3. Architecture globale

## 3.1 Diagramme d'architecture globale

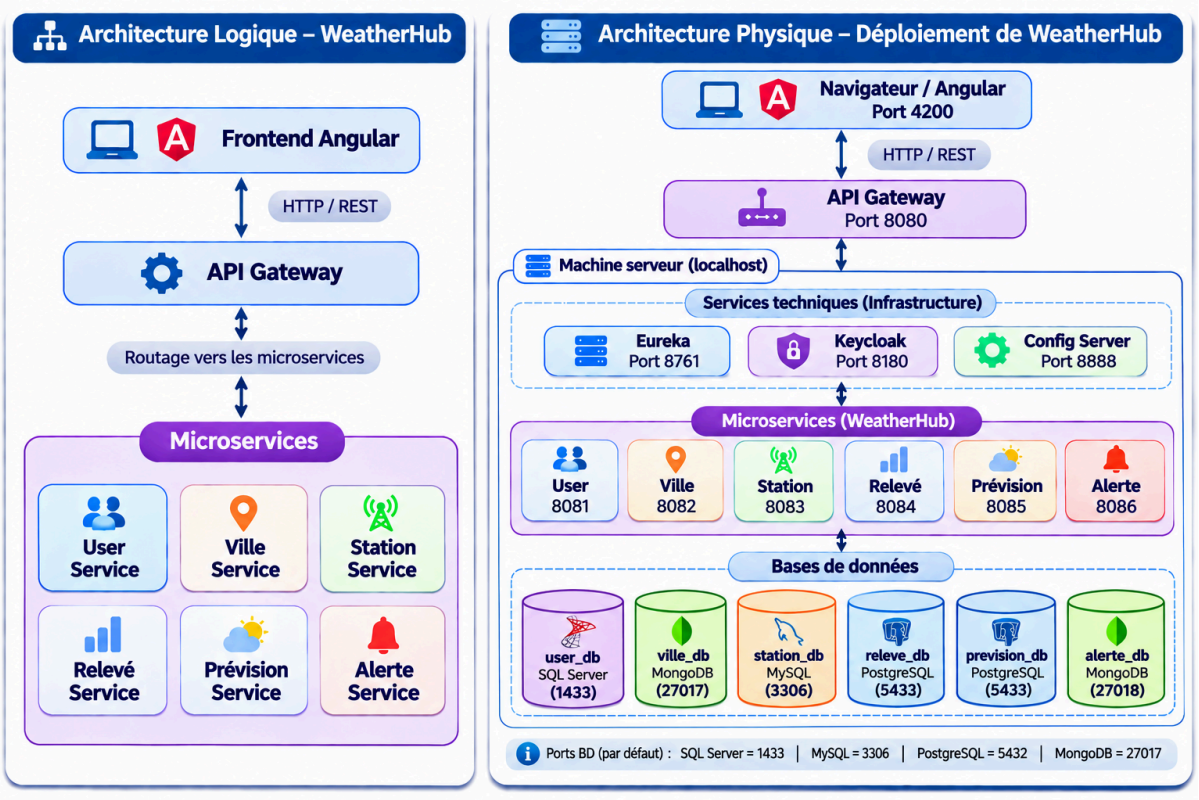



*Figure 2 – Architecture logique (à gauche) et architecture physique de déploiement (à droite)*

## 3.2 Composants

| Composant | Rôle |
|---|---|
| Frontend | Application Angular (port 4200) consommant l'API via la Gateway en HTTP / REST. |
| API Gateway | Point d'entrée unique (port 8080) : routage, validation du JWT, contrôle d'accès par rôle, CORS. Spring Cloud Gateway. |
| Service Discovery | Enregistrement et découverte dynamique des services (Eureka, port 8761). |
| Keycloak | Gestion des identités et des accès (port 8180). |
| Config Server | Configuration centralisée des microservices (port 8888). |
| Microservices métier | Ville, Station, Relevé, Prévision, Alerte, chacun avec sa propre base. |
| Service commun (Auth) | Gestion des utilisateurs, des rôles et émission des JWT. |
| Message broker | Communication asynchrone par événements. |

### 3.3 Principes architecturaux

- **Une base de données par microservice** : aucune clé étrangère entre services. Les références inter-services (villeId, stationId, utilisateurId, responsableId) sont des références logiques par identifiant.
- **Communication synchrone** (REST via OpenFeign) pour les vérifications nécessitant une réponse immédiate.
- **Communication asynchrone** (événements) pour le découplage : mesures, alertes, notifications.
- **Cohérence éventuelle** entre services, avec gestion des suppressions par événements.

## 4. Découpage en microservices

| Service | Responsabilité | Données | Base de données |
|---|---|---|---|
| Ville | Pays, régions, villes, recherche des villes proches (latitude / longitude) | Pays, Region, Ville | MongoDB (ville_db) |
| Station | Stations, capteurs, état (ACTIVE / INACTIVE / MAINTENANCE) | Station, Capteur | MySQL (station_db) |
| Relevé | Mesures, résumés journaliers, historique et statistiques | Mesure, ResumeJournalier | PostgreSQL (releve_db) |
| Prévision | Prévisions sur 7 jours, comparaison prévision / mesure | Prevision, ComparaisonPrevision | PostgreSQL (prevision_db) |
| Alerte | Règles, alertes, notifications, abonnements | Alerte, RegleAlerte, Notification, AbonnementAlerte | MongoDB (alerte_db) |
| Commun (Auth) | Utilisateurs, rôles, authentification JWT | Utilisateur, Role | SQL Server (user_db) |

### 4.1 Justification des choix de bases de données

- **Ville – MongoDB :** permet de stocker facilement des données géographiques avec une structure flexible (pays, régions, villes et coordonnées).
- **Station – MySQL :** adapté aux données structurées et aux relations entre stations et capteurs.
- **Relevé – PostgreSQL :** permet de gérer efficacement les mesures, les historiques et les statistiques grâce à ses capacités de requêtes et d'agrégation.
- **Prévision – PostgreSQL :** adapté au stockage structuré des prévisions et à la comparaison avec les mesures réelles.
- **Alerte – MongoDB :** offre une structure flexible pour gérer les différents types d'alertes, les règles, les notifications et les abonnements.
- **Commun (Auth) – SQL Server :** adapté à la gestion structurée des utilisateurs, des rôles et des relations entre eux, avec des mécanismes de sécurité et d'intégrité des données.

### 4.2 Répartition des services

| Service | Responsable | Port |
|---|---|---|
| Ville | Alaa Ben Moussa | 8082 |

| Service | Responsable | Port |
|---|---|---|
| Station | Mariem Hamdi | 8083 |
| Relevé | Aziz Tombari | 8084 |
| Prévision | Maryem Zayani | 8085 |
| Alerte | Maissa Messoussi | 8086 |
| Commun (Auth) | Toute l'équipe | 8081 (service User) |

# 5. Conception détaillée des microservices

Chaque sous-chapitre présente les données, le diagramme de classes, le contrat d'API, les événements et les interactions du service. Le diagramme de classes global figure en annexe.

## 5.1 Ville Service

**Responsable :** Alaa Ben Moussa

**Responsabilités :** gérer les pays, régions et villes, et fournir la recherche des villes proches d'une position.

### Données gérées et base de données

Entités : **Pays, Region, Ville**.

Base de données : MongoDB (base ville_db, port 27017).

### Contrat d'API

| Endpoint | Méthode | Accès | Codes de réponse |
|---|---|---|---|
| /api/villes | GET | Public / USER | 200 |
| /api/villes/{id} | GET | Public / USER | 200, 404 |
| /api/villes/proches?lat=&lon=&rayon= | GET | Public | 200, 400 |
| /api/villes | POST | ADMIN | 201, 400, 401, 403, 409 |
| /api/villes/{id} | PUT | ADMIN | 200, 400, 401, 403, 404 |
| /api/villes/{id} | DELETE | ADMIN | 204, 401, 403, 404 |

### Événements

**Publiés :** VilleSupprimee.

**Consommés :** aucun.

### Interactions avec les autres services

- Appelé par Station (OpenFeign) pour vérifier l'existence d'une ville.
- Appelé par Prévision pour vérifier la ville d'une prévision.

## 5.2 Station Service

**Responsable :** Mariem Hamdi

**Responsabilités :** gérer les stations et leurs capteurs, ainsi que l'état de chaque station.

### Données gérées et base de données

Entités : **Station, Capteur, enums EtatStation et TypeCapteur**.

Base de données : MySQL (base station_db, port 3306).

### Contrat d'API

| Endpoint | Méthode | Accès | Codes de réponse |
|---|---|---|---|
| /api/stations | GET | USER | 200, 401 |
| /api/stations/{id} | GET | USER | 200, 401, 404 |
| /api/stations | POST | ADMIN | 201, 400, 401, 403, 404, 409 |
| /api/stations/{id} | PUT / DELETE | ADMIN | 200 / 204, 401, 403, 404 |
| /api/stations/{id}/etat | PATCH | ADMIN | 200, 400, 401, 403, 404 |
| /api/stations/{id}/capteurs | GET / POST | USER / ADMIN | 200 / 201, 401, 403, 404 |

*Les endpoints sont une proposition à ajuster selon l'implémentation réelle.*

### Événements

**Publiés :** EtatStationChange.

**Consommés :** VilleSupprimee.

### Interactions avec les autres services

- Appelle Ville pour vérifier la ville à la création d'une station.
- Appelle Auth pour vérifier le responsable (responsableId).
- Appelé par Relevé pour vérifier que la station est ACTIVE.

## 5.3 Relevé Service

**Responsable :** Aziz Tombari

**Responsabilités :** enregistrer les mesures, calculer les résumés journaliers et fournir historique et statistiques.

### Données gérées et base de données

Entités : **Mesure, ResumeJournalier**.

Base de données : PostgreSQL (base releve_db, port 5433).

### Diagramme de classes

### Contrat d'API

| Endpoint | Méthode | Accès | Codes de réponse |
|---|---|---|---|
| /api/mesures | POST | ADMIN | 201, 400, 401, 403, 404 |
| /api/stations/{id}/mesures/derniere | GET | Public | 200, 404 |
| /api/stations/{id}/mesures?debut=&fin= | GET | USER | 200, 400, 401, 404 |
| /api/stations/{id}/statistiques?debut=&fin= | GET | USER | 200, 400, 401, 404 |
| /api/mesures/{id} | PUT / DELETE | ADMIN | 200 / 204, 401, 403, 404 |

*Les endpoints sont une proposition à ajuster selon l'implémentation réelle.*

### Événements

**Publiés :** MesureEnregistree.

**Consommés :** EtatStationChange.

### Interactions avec les autres services

- Appelle Station pour vérifier que la station est ACTIVE avant d'enregistrer une mesure.
- Appelé par Prévision pour la comparaison prévision / mesure.

## 5.4 Prévision Service

**Responsable :** Maryem Zayani

**Responsabilités :** gérer les prévisions sur 7 jours et comparer prévisions et mesures réelles.

### Données gérées et base de données

Entités : **Prevision, ComparaisonPrevision, enum ConditionMeteo**.

Base de données : PostgreSQL (base prevision_db, port 5433).

### Diagramme de classes

### Contrat d'API

| Endpoint | Méthode | Accès | Codes de réponse |
|---|---|---|---|
| /api/villes/{id}/previsions | GET | Public | 200, 404 |
| /api/previsions | POST | ADMIN | 201, 400, 401, 403, 404 |
| /api/previsions/{id} | PUT / DELETE | ADMIN | 200 / 204, 401, 403, 404 |
| /api/villes/{id}/comparaisons?jour= | GET | USER | 200, 400, 401, 404 |

*Les endpoints sont une proposition à ajuster selon l'implémentation réelle.*

### Événements

**Publiés :** PrevisionPubliee (optionnel).

**Consommés :** VilleSupprimee.

### Interactions avec les autres services

- Appelle Ville pour vérifier la ville.
- Appelle Relevé pour obtenir la mesure réelle lors de la comparaison.

## 5.5 Alerte Service

**Responsable :** Maissa Messoussi

**Responsabilités :** gérer les règles d'alerte, générer les alertes, notifier les abonnés et gérer les abonnements.

### Données gérées et base de données

Entités : **Alerte, RegleAlerte, Notification, AbonnementAlerte, enums TypeAlerte, NiveauAlerte, StatutAlerte**.

Base de données : MongoDB (base alerte_db, port 27018).

### Diagramme de classes

### Contrat d'API

| Endpoint | Méthode | Accès | Codes de réponse |
|---|---|---|---|
| /api/alertes/actives | GET | Public | 200 |
| /api/alertes | POST | ADMIN | 201, 400, 401, 403 |
| /api/regles-alerte | GET / POST | ADMIN | 200 / 201, 400, 401, 403 |
| /api/regles-alerte/{id} | PUT / DELETE | ADMIN | 200 / 204, 401, 403, 404 |
| /api/abonnements | POST / DELETE | USER | 201 / 204, 400, 401, 404, 409 |
| /api/notifications | GET | USER | 200, 401 |
| /api/notifications/{id}/lue | PATCH | USER | 200, 401, 404 |

*Les endpoints sont une proposition à ajuster selon l'implémentation réelle.*

### Événements

**Publiés :** AlerteCreee.

**Consommés :** MesureEnregistree, EtatStationChange, VilleSupprimee.

### Interactions avec les autres services

- Consomme MesureEnregistree, évalue les règles, crée l'alerte puis les notifications.
- Appelle Auth pour valider les utilisateurs référencés (utilisateurId).

## 5.6 Service commun (Auth)

**Responsable :** toute l'équipe

**Responsabilités :** gérer les utilisateurs, les rôles et l'authentification par JWT.

### Données gérées et base de données

Entités : **Utilisateur, enum Role (ADMIN, USER)**.

Base de données : SQL Server (base user_db, port 1433).

### Diagramme de classes

### Contrat d'API

| Endpoint | Méthode | Accès | Codes de réponse |
|---|---|---|---|
| /api/auth/register | POST | Public | 201, 400, 409 |
| /api/auth/login | POST | Public | 200, 400, 401 |
| /api/users/me | GET / PUT | USER | 200, 400, 401 |
| /api/users | GET | ADMIN | 200, 401, 403 |
| /api/users/{id} | PUT / DELETE | ADMIN | 200 / 204, 401, 403, 404 |

*Les endpoints sont une proposition à ajuster selon l'implémentation réelle.*

### Événements

**Publiés :** UtilisateurSupprime (optionnel).

**Consommés :** aucun.

### Interactions avec les autres services

- Émet les JWT validés par la Gateway.
- Interrogé par Station et Alerte pour valider les identifiants utilisateurs.

**Ce service est développé par toute l'équipe.**

# 6. Interactions entre services

## 6.1 Communications synchrones

| Appel | Objectif |
|---|---|
| Station ⟶ Ville | Vérifier l'existence de la ville à la création d'une station |
| Station ⟶ Auth | Vérifier le responsable de la station |
| Relevé ⟶ Station | Vérifier que la station est ACTIVE avant d'enregistrer une mesure |
| Prévision ⟶ Ville | Vérifier la ville d'une prévision |
| Prévision ⟶ Relevé | Récupérer la mesure réelle pour la comparaison |

### 6.2 Communications asynchrones

| Événement | Émetteur | Consommateurs et effet |
|---|---|---|
| MesureEnregistree | Relevé | Alerte : évalue les règles et peut créer une alerte |
| AlerteCreee | Alerte | Alerte : génère les notifications pour les abonnés |
| EtatStationChange | Station | Relevé / Alerte : réagissent au changement d'état |
| VilleSupprimee | Ville | Station, Prévision, Alerte : nettoyage des données liées |

## 7. Sécurité

- **Authentification :** login puis émission d'un JWT contenant l'identifiant et le rôle (claims) ; mots de passe hachés avec BCrypt.
- **API Gateway :** routes publiques pour le visiteur, routes protégées par rôle USER ou ADMIN, validation de la signature et de l'expiration du JWT.
- **Services :** Spring Security et contrôle par rôle (@PreAuthorize) pour une défense en profondeur.
- **Autres mesures :** validation des entrées (Bean Validation), CORS, secrets hors du dépôt Git.

| Ressource | Visiteur | USER | ADMIN |
|---|---|---|---|
| Villes, prévisions 7 jours, dernière mesure, alertes actives (lecture) | Oui | Oui | Oui |
| Profil, abonnements, notifications, historique, statistiques | Non | Oui | Oui |
| Gestion des données et des comptes | Non | Non | Oui |

## 8. Technologies choisies

| Domaine | Technologie |
|---|---|
| Backend | Java, Spring Boot, Spring Data JPA |
| Microservices | Spring Cloud Gateway, Eureka, Config Server, OpenFeign |
| Sécurité | Keycloak, Spring Security, JWT |
| Bases de données | SQL Server, MySQL, PostgreSQL, MongoDB |
| Frontend | Angular |
| Documentation d'API | Swagger / OpenAPI |
| Conteneurisation | Docker, Docker Compose |
| Versionnement | Git, GitHub |
| Gestion de projet | *Github* |
| Tests | *Postman* |

# 9. Organisation du dépôt Git et gestion de projet

## 9.1 Structure du dépôt

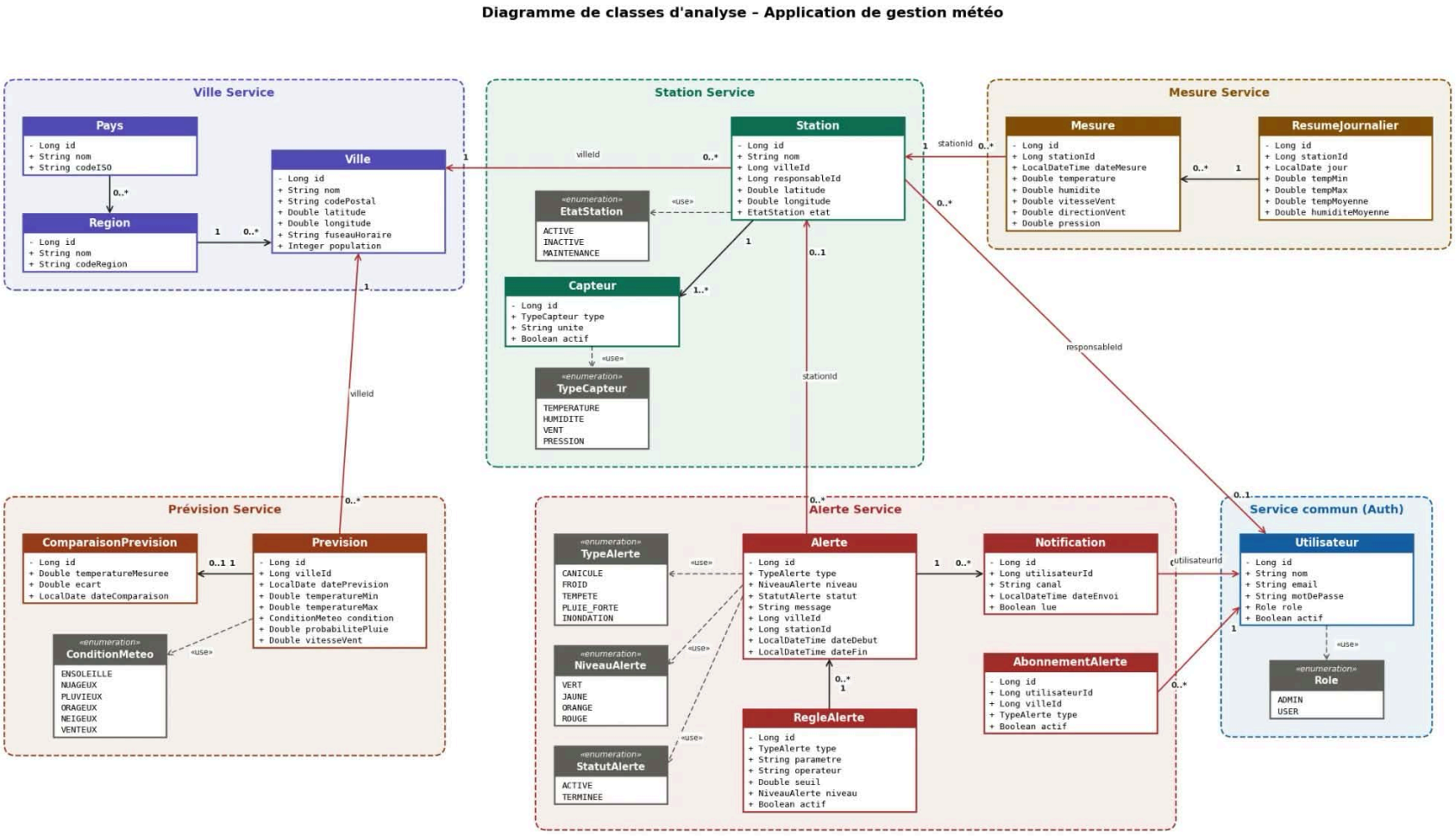
```
meteo-4das16-gX/
├── Backend/            (ville-service, station-service, mesure-service,
│                       prevision-service, alerte-service, auth-service,
│                       api-gateway, eureka-server)
├── Frontend/
├── Documentation/
│   ├── Presentation/
│   └── Diagrams/
└── README.md
```

# 10. Conclusion et perspectives

Ce dossier a présenté la conception de **SkyCast**, une application de gestion météo reposant sur une architecture microservices Spring Boot. Nous avons identifié les acteurs et leurs fonctionnalités, découpé le système en cinq services métier et un service commun d'authentification (chacun avec sa propre base de données), puis défini une architecture faiblement couplée : API Gateway, Eureka, Config Server, sécurité JWT, et communications synchrones (OpenFeign) et asynchrones (événements). Cette conception répond à la problématique posée en offrant une base évolutive, modulaire et sécurisée.

L'implémentation n'étant pas encore finalisée, les prochaines étapes sont : le développement des microservices et l'ajustement des contrats d'API, la mise en place de l'infrastructure et de la conteneurisation (Docker Compose), le développement du frontend Angular, puis la validation par des tests (Postman) et la documentation Swagger. À plus long terme, le projet pourrait évoluer vers l'intégration de capteurs ou de sources météo externes, des notifications multicanaux (e-mail, SMS) et une supervision centralisée des services.

# Annexe A – Diagramme de classes d'analyse global




*Les traits rouges représentent des références logiques inter-services (par identifiant, sans clé étrangère).*